COMPONENT BASED SOFTWARE ENGINEERING

Component-Based Software Design

What is Component Design?

- Component design is all about taking complex software systems and making them into small, reusable pieces or simply modules. These parts are responsible for directing certain functionalities, so programming them is like building a puzzle with small pieces, which eventually create more complex architectures. Component design exhibits a modularized approach to software development where the units are organized systematically to facilitate control of complexity and increase manageability.
- Elements being the fundamental building blocks of software architecture, they are mostly responsible for enriching the software's functionality or offering their services.
- When several features are executed in independent units called components, with this approach modularity, reusability, and maintainability are ensured.
- This technique promotes flexibility and scalability, enabling the formation of dynamic systems that are composed of more basic units that interlock each other.

Importance of Component-Based Design in Software Development

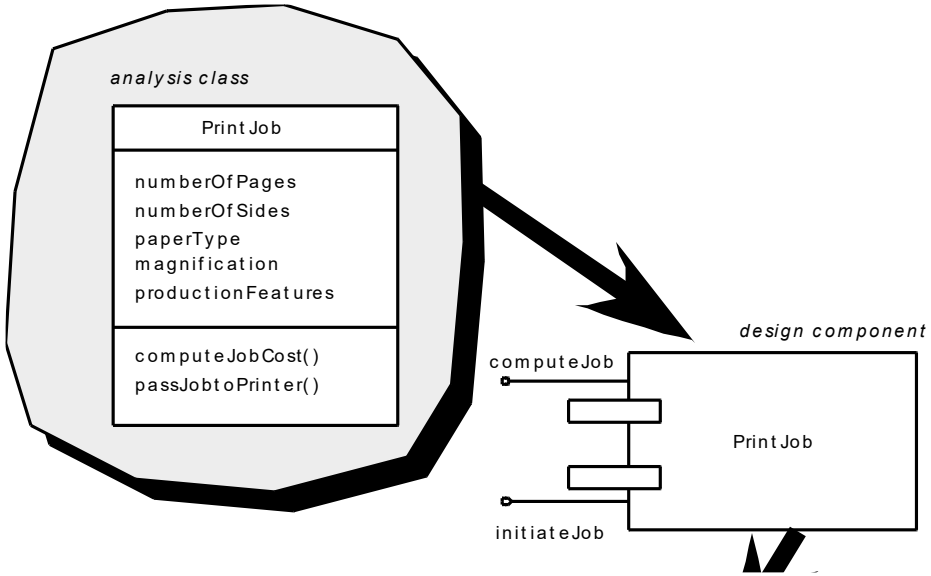
- Component-based design leads to various beneficial outcomes at all the development life stages of the software.
- It improves modularity, giving the possibility for all developers to work with different parts separately without changing the whole system. This modularity enables code reusability since components can be reused across several projects as this will save time and effort.
- Component-based design is so important for maintainability as it results from being broken down into different parts and depicts functionality, with which errors are easily traced, and debugging, and testing are made possible.

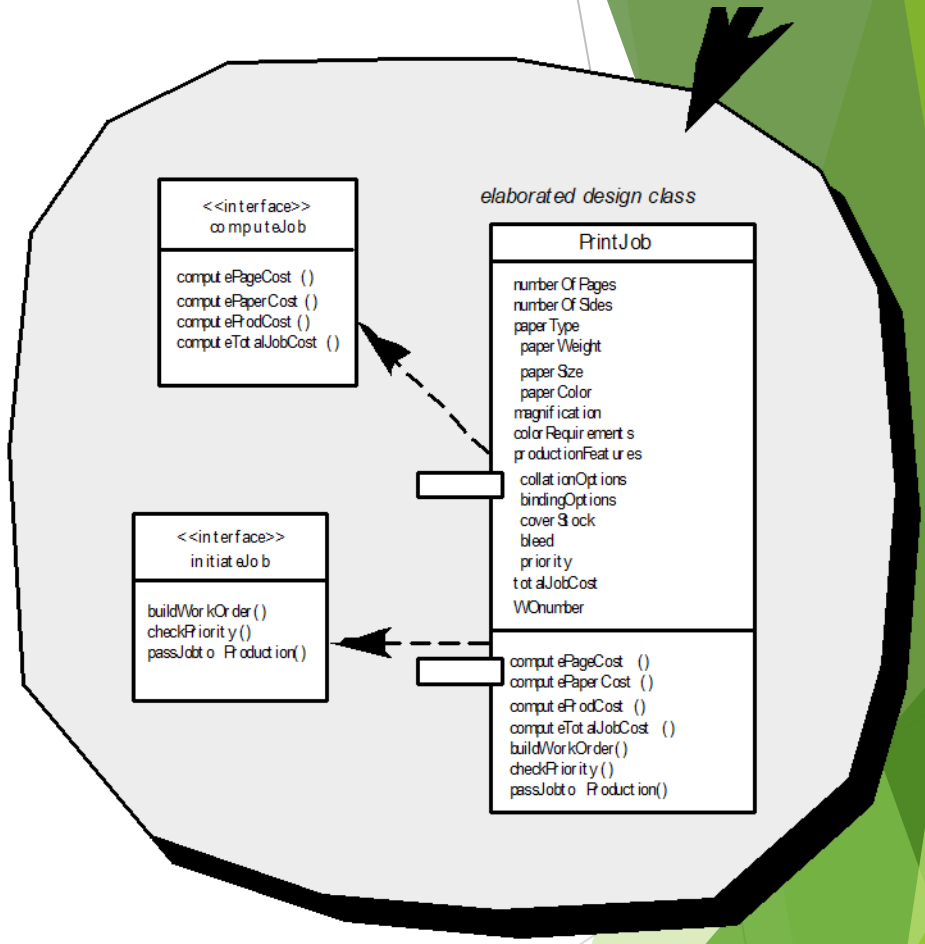
Cont.: Importance of Component-Based Design in Software Development

- When it comes to scalability, component-based design forms another vital benefit. A situation when a particular software system, as it turns out to be more complex, has to be scalable enough, leads to making this ability the priority.
- Breaking down elements and establishing logical interfaces facilitate the creation of a modular system that permits effortless integration of more capabilities without causing the architecture to buckle. It highlights the fact that adaptable software systems can not only be quickly modified but also evolve according to the changing requirements.

An Object-Oriented View

- Each class within a component has been fully elaborated to include all attributes and operations that are relevant to its implementation.
- As part of the design elaboration, all interfaces (messages) that enable the classes to communicate and collaborate with other design classes must also be defined.
- To accomplish this, the designer begins with the analysis model and elaborates analysis classes (for components that relate to the problem domain) and infrastructure classes (or components that provide support services for the problem domain).


analysis class
PrintJob
numberOfPages
numberOfSides
paperType
magnification
productionFeatures
computeJobCost()
passJobtoPrinter()
design component
computeJob
PrintJob
initiateJob


<<interface>>
computeJob
computePageCost ()
computePaperCost ()
computeProdCost ()
computeTotalJobCost ()
elaborated design class
PrintJob
numberOfPages
numberOfSides
paperType
paperWeight
paperSize
paperColor
magnification
colorRequirements
productionFeatures
collationOptions
bindingOptions
coverStock
bleed
priority
totalJobCost
WOnumber
computePageCost ()
computePaperCost ()
computeProdCost ()
computeTotalJobCost ()
buildWorkOrder()
checkPriority()
passJobtoProduction()
<<interface>>
initiateJob
buildWorkOrder()
checkPriority()
passJobtoProduction()

Cont: Conventional view:

- Conventional view: a component is a functional element of a program that incorporates processing logic, the internal data structures that are required to implement the processing logic, and an interface that enables the component to be invoked and data to be passed to it.
- A conventional component, also called a *module*, resides within the s/w architecture and serves one of three important roles as:

Roles

1. A *control component* that coordinates the invocation of all other problem domain components,
2. A *problem domain component* that implements a complete or partial function that is required by the customer,
3. An *infrastructure component* that is responsible for functions that support the processing required in the problem domain.

Component level design has two main goals:

1. Identify the components that are needed to build the system.

- This includes determining the boundaries of each component and how they relate to one another.

2. Define the interfaces between components.

- This makes the components loosely coupled and independent, allowing them to be developed and tested separately before being integrated into the full system.

When designing components, software engineers consider:

- **Functionality**: What is the purpose of the component? What features will it provide?
- **Data**: What data does the component need to operate? What data does it produce? How will it interface with data from other components?
- **Dependencies**: On what other components does this component rely? How will those dependencies be handled?
- **Reusability**: Can this component be reused in other systems? If so, how can its design be made more generic?
- **Scalability**: How will this component handle increases in load or volume? Can its performance be optimized?

Note:

- Component-level design is a fundamental step in building a robust, modular software architecture.
- By determining how to break a complex system into discrete, interconnected parts, software engineers can create solutions that are flexible, extensible, and resilient to changes in requirements.
- The end result is a system that is greater than the sum of its parts.

Designing Class-Based Components

- These principles can be used to guide the designer as each S/W component is developed.
- **The Open-Closed Principle (OCP).** *"A module [component] should be open for extension but closed for modification.* The designer should specify the component in a way that allows it to be extended without the need to make internal (code or logic-level) modifications to the component.
- **The Liskov Substitution Principle (LSP).** *"Subclasses should be substitutable for their base classes.* A component that uses a base class should continue to function properly if a class derived from the base class is passed to the component instead.
- **Dependency Inversion Principle (DIP).** *"Depend on abstractions. Do not depend on concretions."* Abstractions are the place where a design can be extended without great complications.

Cont.:

- **The Interface Segregation Principle (ISP).** *"Many client-specific interfaces are better than one general purpose interface.* The designer should create a specialized interface to serve each major category of clients.
- **The Release Reuse Equivalency Principle (REP).** *"The granule of reuse is the granule of release."* When classes or components are designed for reuse, there is an implicit contract that is established between the developer of the reusable entity and the people who will use it.
- **The Common Closure Principle (CCP).** *"Classes that change together belong together."* Classes should be packaged cohesively. When some characteristic of an area must change, it is likely that only those classes within the package will require modification.
- **The Common Reuse Principle (CRP).** *"Classes that aren't reused together should not be grouped together."* When one or more classes with a package changes, the release number of the package changes.

Component-Level Design Guidelines

- **Components**
  - Naming conventions should be established for components that are specified as part of the architectural model and then refined and elaborated as part of the component-level model
- **Interfaces**
  - Interfaces provide important information about communication and collaboration (as well as helping us to achieve the OCP)
- **Dependencies and Inheritance**
  - It is a good idea to model dependencies from left to right and inheritance from bottom (derived classes) to top (base classes).

Cohesion

- *Cohesion* implies that a component or class encapsulates only attributes and operations that are closely related to one another and to the class or component itself.
- Levels of cohesion
  - Functional: occurs when a module performs one and only one computation and then returns a result.
  - Layer: occurs when a higher layer accesses the services of a lower layer, but lower layers do not access higher layers.

Cont.: Cohesion

- Communicational: All operations that access the same data are defined within one class.
- Sequential: Components or operations are grouped in a manner that allows the first to provide input to the next and so on.
- Procedural: Components or operations are grouped in a manner that allows one to be invoked immediately after the preceding one was invoked.
- Temporal: Operations that are performed to reflect a specific behavior or state.
- Utility: Components, classes, or operations that exist within the same category but are otherwise unrelated are grouped together.

Coupling

- Conventional view:
  - The degree to which a component is connected to other components and to the external world
- OO view:
  - A qualitative measure of the degree to which classes are connected to one another. Keep coupling as low as possible.
- Level of coupling

Level of coupling

- **Content**: Occurs when one component “superstitiously modifies data that is internal to another component. “Violates Information hiding”
- **Common**: Occurs when a number of components all make use of a global variable.
- **Control**: Occurs when *operation A()* invokes *operation B()* and passes a control flag to *B*. The control flag then “directs” logical flow within *B*.
- **Stamp**: Occurs when **ClassB** is declared as a type for an argument of an operation of **ClassA**. Because **ClassB** is now a part of the definition of **ClassA**, modifying the system becomes more complex.

Cont.:

- **Data**: Occurs when operations pass long strings of data arguments. “Testing and maintenance becomes more difficult.”
- **Routine call**: Occurs when one operation invokes another.
- **Type use**: Occurs when component **A** uses a data type defined in component **B.**
- **Inclusion or import:** Occurs when component **A** imports or includes a package or the content of component **B.**
- External: Occurs when a component communicates or collaborates with infrastructure components (O/S function).

Cont.: Conducting Component-Level Design

- The steps discussed in this section provide a reasonable task set for designing a component. You should emphasize that

(1) Design classes in the problem domain are usually custom-designed, however, if an organization has encouraged design for reuse, there may be an existing component that fits the bill;

(2) Design classes corresponding to the infrastructure domain can sometimes be often from existing class libraries;

3) A UML collaboration diagram provides an indication of message passing between components.

Steps

- Step 1. Identify all design classes that correspond to the problem domain.
- Step 2. Identify all design classes that correspond to the infrastructure domain.
- Step 3. Elaborate all design classes that are not acquired as reusable components.
- Step 3a. Specify message details when classes or component collaborate.
- Step 3b. Identify appropriate interfaces for each component.
- Step 3c. Elaborate attributes and define data types and data structures required to implement them.
- Step 3d. Describe processing flow within each operation in detail.

Cont.:

- Step 4. Describe persistent data sources (databases and files) and identify the classes required to manage them.
- Step 5. Develop and elaborate behavioral representations for a class or component.
- Step 6. Elaborate deployment diagrams to provide additional implementation detail.
- Step 7. Factor every component-level design representation and always consider alternatives.

Component Lifecycle Management

Component Lifecycle Management is a systematic approach to managing the lifecycle of software components from their creation to decommissioning.

Stage 1: Creation: Components either get initialized statically or dynamically at design time based on the needs of the program and according to the entire program or system architecture.

Stage 2: Initialization: Components are set up, configured and prepared for use, usually this include parameterization, allocation of resources and the side setup.

Cont.: Component Lifecycle Management

- **Stage 3: Utilization:** Components are utilized within the software system, fulfilling their designated functionalities and interacting with other components as necessary.
- **Stage 4: Maintenance:** Components undergo maintenance activities such as updates, bug fixes, and optimizations to ensure continued functionality and performance.
- **Stage 5: Decommissioning:** Components are decommissioned, or disposed, when they are no longer needed, either from obsolescence, redundancy, or architectural changes.

Characteristics of Component-Based Design

Below are the characteristics of component-based design:

- **Modularity:** Some parts of a program retain a certain function or service, essentially serve as reusable software modules, interchangeable and therefore independent components, enhancing flexibility in development, testing, and maintenance.
- **Reusability:** Component architectural features are envisioned to be adaptable to and reusable on different projects giving rise to shorter development time and lower overheads and making the work of developers much more easier and scientific.

Characteristics of Component-Based Design

- **Interoperability:** Components are connected through clearly defined interfaces, ensuring that software systems maintain interaction continuity and carry out various functions within their ecosystems.
- **Encapsulation:** Components have got encapsulated constructs, which precisely reveal internal details at the interface level, through a provision of only a few interfaces essential for interacting with other components, thus enforcing abstraction by hiding underlying implementation details.
- **Scalability:** Component-based architectures simplify scalability by providing a way for systems to grow organically via the addition or modification of components without the impacting overall the architecture.

Types of Components

- **UI Components**
  - User Interface components provide an easy and more convenient way to encapsulate logic by combining presentational and visible elements such as buttons, forms, and widgets.
- **Service Components**
  - Service components are the base of business logic or application services, in which they serve as the platform for activities such as data processing, authentication, and communication with external systems.
- **Data Components**
  - Through data abstraction and provision of interfaces for data access, data components take care of database interaction issues and provide data structures for querying, updating, and saving data.

Cont.:

- **Infrastructure Components**
  - The hardware elements regard as fundamental services or resources like logging, caching, security and communication protocols which a software system depends on.
- **Integration Components**
  - Integrated components for data communication and data exchange between different systems or modules are the integration components, which enable protocol translation, workflow orchestration, and data exchange.
- **Reusable Components**
  - A reusable component, in turn, encapsulates common functionality or algorithms that can be utilized across multiple projects as well as different domains, which promotes code reuse and uniformity.

Component Design Patterns

- Component design patterns present the designer with patterns that can be reused to solve typical problems that design architects face in the construction of component-based architectures Such patterns carry guidance on the effective design and implementation of components as well as the art of integrating these effectively. Some popular component design patterns include:
- **Composite Pattern:** The Composite pattern allows objects to be composed into tree structures, treating individual objects and compositions of objects uniformly.
- **Decorator Pattern:** The decorator pattern provides the facility of dynamic enhancement in the object behavior by wrapping decorators that are added to existing functionalities without changing the interface.

Cont.: Component Design Patterns

- **Adapter Pattern:** Through Adapter pattern, interface incompatibility issues are solved as a bridge is built that translates requests from one interface to another by using this one.
- **Factory Pattern:** Factory pattern comprehends the process of preparing objects, with the outcome of providing customers the ability to create objects without specifying their concrete classes, hence facilitating flexibility and decoupling.
- **Observer Pattern:** The Observer pattern constitutes a unidirectional link between objects, a one-to-many dependency, where objects notifying and updating other objects automatically, as their state changes.
- **Facade Pattern:** The Facade pattern provides a simplified interface to a complex system, encapsulating its intricate details and presenting a unified interface to clients.
- **Singleton Pattern:** The Singleton pattern, which ensures that a class has only one instance and grants global access to that object, enables one to control centralized access to shared objects and data.

Tools and Technologies for Component Design

- **Component Frameworks:** Components frameworks for instance, Angular, React, and Vue.js include a variety of pre-built components, templates, and utilities for building user interfaces that users can interact.
- **Dependency Injection Containers:** Injection Containers of IoC nature like Spring IoC, Guice and Dagger can take the control of inversion by managing the component dependencies as well as the producing the instances for the dependent classes.
- **Middleware Platforms:** API middleware systems like Apache Camel or MuleSoft or RabbitMQ connect different system and services by means of message routing, transformation, and mediation.
- **Component Repositories:** These repositories like npm, Maven Central, and NuGet pool together the storage, sharing, and discovery of libraries and reusable components.
- **Component Testing Tools:** Tools like Jest, JUnit, and Mockito are used for automated testing of components; this lets their functionality, reliability and adherence to specification to be verified.

Common Mistakes in Component Level Design

(i) Lack of Cohesion

- Cohesion refers to the degree to which the elements of a component belong together.
- A cohesive component has a singular, focused purpose and contains closely related elements.
- Lack of cohesion is one of the most common mistakes in component level design. It often results in components that are difficult to understand, reuse, and maintain.

Cont.:

(ii) Excessive Coupling

- Coupling refers to the degree of interdependence between components.
- Excessively coupled components are too dependent on each other, making them hard to reuse and maintain independently.
- Loose coupling, on the other hand, reduces interdependencies and allows components to operate independently.
- Excessive coupling is a frequent mistake that contradicts the goals of component-based design.

Cont.:

(iv) Violating Encapsulation

- Encapsulation means hiding the internal details and implementation of a component from the outside.
- It enables the internal workings of a component to change without impacting other components.
- Failing to properly encapsulate components exposes their inner workings, creating tight coupling and dependencies on implementation details that may change.
- This mistake compromises the flexibility and maintainability of the overall system.

Cont.:

(v) Lack of Abstraction

- Abstraction involves suppressing irrelevant details about a component in order to reduce complexity and facilitate understanding. Lack of abstraction results in low-level components that are overly complex, difficult to understand, and reuse. Effective abstraction focuses on the essential purpose and functionality of a component, hiding unnecessary details.

(vi) Not Considering Reusability

- If components are not designed with reuse in mind, the system will end up with many redundant components that cannot be reused in other contexts. This error leads to increased development and maintenance efforts. On the other hand, reusable components can be used in multiple systems, reducing their development cost

Challenges of Component Design

Below are the challenges of component design:

- **Granularity:** Proper level of granularity is what designers of components need to choose accurately, as excessively fine components tend to clutter and increase overhead, while coarse grained ones are inappropriately inflexible and have a limited ability to reuse.
- **Interoperability:** Maintaining the seamless operation of components from unmixed vendors or technologies may be a difficult task. For this reason, standardized interfaces, protocols and compatibility tests have to be carried out.
- **Dependency Management:** Keeping dependencies between the components, such as versions, compatibility, and conflicts can be extremely difficult especially in the bigger systems, which have a augmented number of dependencies.
- **Performance Overhead:** The component-based architectures may produce performance overhead for its added levels of abstraction, communication and runtime dependency which require meticulous optimization and profiling thus may result in runtime performance issues.

End

!!!!!!!